

KDP PUZZLE PLATFORM

Architecture, Uniqueness Engine & Product Roadmap

Comprehensive Technical & Product Reference

A complete guide to building a uniqueness-guaranteed, KDP-ready puzzle book generation platform.

Table of Contents

Contents

1 Technology Stack	4
1.1 Infrastructure Overview	4
2 Uniqueness Engine	4
2.1 Three Levels of Uniqueness	4
2.2 Chosen Approach: Seed Registry + Hash Registry	4
2.3 Detailed Implementation Steps	5
2.3.1 Step 1 — Seed Design	5
2.3.2 Step 2 — Deterministic Puzzle Generation	5
2.3.3 Step 3 — Normalization & Hashing	5
2.3.4 Step 4 — Uniqueness Check Flow	6
2.3.5 Step 5 — Database Schema	6
2.3.6 Step 6 — Reservation System (Race Condition Prevention)	6
2.3.7 Step 7 — Pre-Generation Workers	7
2.3.8 Step 8 — Stale Reservation Cleanup	7
2.4 Uniqueness Per Level — Architecture	7
2.4.1 Book-Level Uniqueness	7
2.4.2 User-Level Uniqueness	7
2.4.3 Global-Level Uniqueness	8
3 Full System Architecture Flow	9
3.1 Background Worker Flow (Pre-Generation)	9
3.2 User Request Flow (Book Generation)	9
4 Feature Roadmap	10
4.1 V1 — MVP (Ship It)	10
4.2 V2 — Grow It	10
4.3 V3 — Scale It	10
5 Competitor Analysis	12
5.1 KDPForge	12
5.2 BooksGenie.ai	12
5.3 KDP Tools Pro	12
5.4 Instant Puzzle Generator / PuzzlePro	13
5.5 Free Generators (puzzlegenerators.com etc.)	13
5.6 Competitive Positioning Summary	13
6 Marketing & User Acquisition	14
6.1 Early Acquisition Strategy	14
6.2 Marketing Channels	14
7 Pricing Strategy	14
7.1 V1 — One-Time Purchase	14
7.2 V2 — Subscription (Per Puzzle Type)	14
8 Quick Reference	14

8.1	Uniqueness Level Decision Guide	15
8.2	V1 Build Checklist	15

1 Technology Stack

The platform is built on a modern, scalable stack chosen for developer familiarity, SEO capability, and cost efficiency on a VPS.

Layer	Technology & Reason
Frontend	Next.js — SSR/SSG for SEO, fast page loads, React ecosystem
Backend API	NestJS — Modular, TypeScript-first, excellent for structured APIs
Database	PostgreSQL — Reliable, supports <code>SKIP LOCKED</code> for job queues
Job Queue	BullMQ + Redis — Async puzzle generation workers
Caching	Redis — Hash lookups, reservation TTLs, rate limiting
Payments	LemonSqueezy — Simple setup, handles VAT, webhooks
PDF Generation	pdf-lib / PDFKit — KDP-valid PDF output
File Storage	Local VPS disk (V1) → S3-compatible object storage (V2+)
Hosting	2 vCPU, 4 GB RAM VPS — sufficient for MVP traffic

1.1 Infrastructure Overview

```

Next.js (Frontend + SSR) <--> NestJS API <--> PostgreSQL
                        |
                        Redis (Cache + Queue)
                        |
                    BullMQ Workers (Puzzle Gen)
                        |
                    PDF Storage (Local --> S3 later)
  
```

2 Uniqueness Engine

The platform supports three levels of uniqueness, each serving a different scope. Users select the level that fits their publishing needs.

2.1 Three Levels of Uniqueness

Level	Scope	Who It's For
Book-Level	No duplicate puzzles within a single book	All users — enforced by default
User-Level	No repeated puzzles across all books by one user	Serious publishers with multiple volumes
Global-Level	No duplicates across the entire platform	Premium subscribers — strong guarantee

2.2 Chosen Approach: Seed Registry + Hash Registry

After evaluating all deduplication strategies, Seed + Hash is the optimal choice at every stage of the platform.

Why This Combination Works

- **Seed Registry:** Every puzzle is generated from a deterministic integer seed. Seeds are never reused.
- **Hash Registry:** Every puzzle grid is SHA-256 hashed and stored. Before delivery, the hash is checked.
- **Together:** Seeds give reproducibility (reprint any puzzle on demand); hashes give collision detection.
- Isomorphic deduplication is **not** included — it is computationally expensive and invisible to human solvers.

2.3 Detailed Implementation Steps

2.3.1 Step 1 — Seed Design

Seeds are structured 64-bit integers that encode difficulty and scope, not random numbers.

```
Seed Structure (64-bit integer):
  Bits 63-62 --> Difficulty tier (00=Easy, 01=Medium, 10=Hard, 11=Expert)
  Bits 61-54 --> Shard / region (for future horizontal scaling)
  Bits 53-0  --> Auto-increment sequence counter
```

Example:

```
Easy   seq 1 --> 0000 0000 0000 ... 0001
Medium seq 1 --> 0100 0000 0000 ... 0001
Hard   seq 1 --> 1000 0000 0000 ... 0001
```

- Seeds are treated like auto-increment primary keys — never reused
- A `seed_counters` table in PostgreSQL tracks the next available seed per difficulty
- The seed counter is the source of truth, not randomness

2.3.2 Step 2 — Deterministic Puzzle Generation

The seed drives a deterministic backtracking algorithm. The same seed always produces the same puzzle.

```
function generatePuzzle(seed: number, difficulty: Difficulty): PuzzleResult {
  const rng      = seededRandom(seed);
  const solution = generateFullGrid(rng);
  const puzzle   = removeClues(solution, difficulty, rng);
  return { seed, puzzle, solution, difficulty };
}
// Clue counts: Easy 36-45 | Medium 27-35 | Hard 22-26 | Expert 17-21
```

2.3.3 Step 3 — Normalization & Hashing

Before hashing, normalize the puzzle grid to a canonical string for consistent results.

Normalization Rules

1. Represent empty cells as "0" (never "." or space)
2. Flatten to a single 81-character string, no separators
3. No whitespace, no newlines

4. Hash only the **puzzle** grid — not the solution, not metadata

Example: "530070000600195000098000060800060003..." (81 chars)

Hash: SHA-256(normalizedString) → 64-char hex digest

2.3.4 Step 4 — Uniqueness Check Flow

```

1. Generate puzzle from seed
2. Normalize grid --> 81-char string
3. SHA-256 hash the string
4. Query: SELECT 1 FROM puzzle_registry WHERE hash = $1

    Hash found?      --> Increment seed counter, regenerate from Step 1
    Hash not found? --> Proceed to Step 5 (reserve + store)

Redis cache sits in front of Postgres for fast lookups.
Target cache hit rate: >95%
```

2.3.5 Step 5 — Database Schema

```

-- Seed counters (one row per difficulty)
CREATE TABLE seed_counters (
    difficulty    VARCHAR(10) PRIMARY KEY,
    next_seed     BIGINT NOT NULL DEFAULT 1
);

-- Main puzzle registry
CREATE TABLE puzzle_registry (
    id            BIGSERIAL PRIMARY KEY,
    seed          BIGINT NOT NULL UNIQUE,
    hash          CHAR(64) NOT NULL UNIQUE,    -- SHA-256
    difficulty    VARCHAR(10) NOT NULL,
    user_id       UUID REFERENCES users(id),
    book_id       UUID REFERENCES books(id),
    status        VARCHAR(12) DEFAULT 'available',
    reserved_at   TIMESTAMP,
    created_at    TIMESTAMP DEFAULT NOW()
);

CREATE UNIQUE INDEX idx_puzzle_hash ON puzzle_registry(hash);
CREATE INDEX idx_puzzle_status_diff ON puzzle_registry(status, difficulty);
CREATE INDEX idx_puzzle_user ON puzzle_registry(user_id);
```

2.3.6 Step 6 — Reservation System (Race Condition Prevention)

When multiple users generate books simultaneously, puzzles must be atomically reserved.

Status	Meaning
available	In the pool, ready to be assigned
pending	Reserved for a user — TTL 30 minutes
confirmed	Assigned to a finalized book
expired	Reservation timed out — returned to available

```

UPDATE puzzle_registry
SET
    status = 'pending',
    reserved_at = NOW(),
    user_id = $1
WHERE id IN (
```

```

SELECT id FROM puzzle_registry
WHERE  status = 'available'
AND    difficulty = $2
LIMIT $3
FOR UPDATE SKIP LOCKED
)
RETURNING id, seed, hash;

```

2.3.7 Step 7 — Pre-Generation Workers

Puzzles are generated in background workers before users request them, ensuring zero latency at generation time.

Parameter	Value
Target pool size per difficulty	100,000 puzzles
Worker trigger threshold	Pool drops below 20% (20,000 puzzles)
Batch generation size	5,000 puzzles per worker run
Worker technology	BullMQ + Redis (NestJS Bull module)
Generation speed estimate	~1,000–5,000 puzzles/min per core

2.3.8 Step 8 — Stale Reservation Cleanup

```

-- Runs every 5 minutes via BullMQ scheduled job
UPDATE puzzle_registry
SET    status = 'available',
       user_id = NULL,
       reserved_at = NULL
WHERE  status = 'pending'
AND    reserved_at < NOW() - INTERVAL '30 minutes';

```

2.4 Uniqueness Per Level — Architecture

2.4.1 Book-Level Uniqueness

Prevents duplicates within a single book. Enforced for all users at no extra cost.

1. User requests a book of N puzzles with a given difficulty
2. System reserves N puzzles using `FOR UPDATE SKIP LOCKED`
3. All N puzzles are tagged with the same `book_id`
4. Hash comparison within the batch confirms no intra-book duplicates
5. On export, status is set to `confirmed`

2.4.2 User-Level Uniqueness

Prevents the same puzzle appearing in any two books by the same user. Available on paid plans.

1. All puzzles generated by a user are stored with their `user_id`
2. New puzzle requests filter out puzzles already assigned to that user
3. Users get a “puzzle history” dashboard showing all previously used hashes

```

SELECT id FROM puzzle_registry
WHERE  status = 'available'
AND    difficulty = $difficulty
AND    id NOT IN (

```

```
SELECT puzzle_id FROM user_puzzle_history WHERE user_id = $userId
)
LIMIT $count
FOR UPDATE SKIP LOCKED;
```

2.4.3 Global-Level Uniqueness

Guarantees no puzzle has ever been delivered to any user on the platform. Premium feature.

1. The `status` column becomes the source of truth globally
2. Once confirmed for any user, a puzzle is permanently removed from the pool
3. The global pool is replenished continuously by background workers
4. Users see a “globally unique” badge on their generated books
5. System alerts when global pool drops below 500,000 puzzles

Level	Query Scope	Storage	Latency
Book	Batch of N puzzles	Minimal	None
User	All books by user	Low (index on <code>user_id</code>)	Negligible
Global	Entire puzzle table	High — grows with platform	Low with Redis

3 Full System Architecture Flow

3.1 Background Worker Flow (Pre-Generation)

```
[BullMQ Scheduler] -- runs when pool < 20%
|
[Get next seed] <-- seed_counters table (atomic increment)
|
[Generate puzzle] -- deterministic backtracking from seed
|
[Normalize grid] -- 81-char string, blanks = "0"
|
[SHA-256 hash]
|
[Redis cache check] --> Hit: skip seed, increment, retry
|
[Postgres hash check] --> Exists: skip, retry
|
[INSERT into puzzle_registry] -- status = "available"
|
[Cache hash in Redis]
```

3.2 User Request Flow (Book Generation)

```
[User selects: difficulty, count, uniqueness level, layout]
|
[API validates plan -- does user's plan include this level?]
|
[Reserve N puzzles atomically -- FOR UPDATE SKIP LOCKED]
  * Book-level: filter by difficulty
  * User-level: filter by difficulty + exclude user history
  * Global-level: filter by difficulty + status = available only
|
[Status --> "pending" | TTL = 30 minutes]
|
[Return low-res preview to browser]
|
[User confirms / purchases]
|
[BullMQ PDF generation job created]
|
[PDF worker renders KDP-valid PDF]
|
[PDF stored to disk (V1) or S3 (V2)]
|
[Status --> "confirmed" | book_id assigned]
|
[User gets download link -- expires in 7 days]
```

4 Feature Roadmap

4.1 V1 — MVP (Ship It)

Focus: Get paying users. Prove the concept. Nail quality and KDP compatibility.

Feature	Notes
Email + Google Auth	NextAuth.js — fast setup
Sudoku (Easy / Medium / Hard / Expert)	Deterministic backtracking, seed-based
Single page, multi-page, full book modes	Flexible output for different publishers
KDP PDF (6×9, 8×10, 8.5×11)	Correct bleed, margins, DPI from day one
Answer key pages	Auto-generated — huge time saver
Book-level uniqueness engine	No duplicates within a single book
Seed + hash registry	Foundation for all uniqueness levels
Low-res in-browser preview	V1 differentiator vs competitors
Async generation with BullMQ	API stays responsive under load
LemonSqueezy one-time purchase	Simple, no subscription complexity yet
PDF expiry + auto-cleanup (7 days)	Keeps storage lean on VPS
Basic puzzle history per user	Seeds + hashes stored per account
SEO landing page + blog section	Organic traffic from day one
Public shareable puzzle preview links	Viral loop — publishers share with readers

4.2 V2 — Grow It

Focus: Retention, recurring revenue, power-publisher features.

Feature	Notes
Word search generator	Second puzzle type — expands TAM
Subscription model (per puzzle type)	Pay only for what you use
Book series (Vol. 1–N, no repeats)	Key selling point for prolific publishers
Large print mode	Huge underserved market: elderly, visually impaired
Custom book metadata	Title, author, intro page, back cover text
Difficulty progression	Books that ramp from easy to expert
User-level uniqueness engine	No repeats across all user's books
Puzzle regeneration control panel	Swap one puzzle without regenerating whole book
Bulk puzzle generation (100–500 at once)	Power user feature
Analytics dashboard	Books generated, downloads, popular types

4.3 V3 — Scale It

Focus: Market leadership, unique features no competitor has.

Feature	Notes
Crossword puzzles	High demand — complex to build right
3D word search	Real differentiator — no competitor has this
Themed puzzle books	Holiday, kids, travel, sports themes
Global uniqueness engine	Platform-wide hash — premium guarantee
Bulk generation API (50 books)	Power publishers and agencies
White-label plan	Agencies resell under their own brand
Kakuro, Nonogram, Number Fill	Expand puzzle library
Puzzle marketplace	Users sell puzzle packs to other users
Team accounts	Publishing teams with shared libraries
Affiliate program	KDP communities spread the word

5 Competitor Analysis

5.1 KDPForge

Attribute	Detail
Pricing	\$19.99/mo \$199/yr \$399 lifetime
Key Strength	410+ puzzle types, unlimited generation, bulk export
Key Weakness	Cluttered UI, no uniqueness guarantee, quantity over quality
Opportunity	Win on quality, uniqueness engine, and cleaner UX

Pros	Cons
410+ puzzle types	Low quality across many types
Unlimited generation	Cluttered UI
Bulk PDF export	No uniqueness guarantee
Commercial license	Not specialized
KDP-ready templates	Poor puzzle differentiation

5.2 BooksGenie.ai

Attribute	Detail
Pricing	Starter \$9.99/mo Pro \$15.99/mo Business \$29.99/mo
Key Strength	AI word list generation, integrated cover gen, book builder
Key Weakness	Daily generation limits, low customization, puzzles often repeated
Opportunity	Mathematically correct puzzles + genuine uniqueness beats them

Pros	Cons
AI generates word lists	Daily generation limits
Many puzzle types	Subscription locked
Integrated book builder	Low customization
Cover generation included	Poor quality control

5.3 KDP Tools Pro

Attribute	Detail
Pricing	\$45 lifetime
Key Strength	Very cheap lifetime license, 300 DPI export
Key Weakness	Word search only, weak UI, no automation, no uniqueness engine
Opportunity	Easy to dominate — low bar to beat

Pros	Cons
Very cheap lifetime price 300 DPI export Commercial license	Word search only Weak UI No automation or uniqueness

5.4 Instant Puzzle Generator / PuzzlePro

Attribute	Detail
Pricing	£12/mo Starter £25/mo Pro
Key Strength	Preview before download — match this in V1
Key Weakness	Word search only, limited puzzle library, no advanced puzzles
Opportunity	Sudoku quality + preview = better product from day one

5.5 Free Generators (puzzlegenerators.com etc.)

Attribute	Detail
Pricing	Free
Key Strength	Zero cost — attracts casual users
Key Weakness	Outdated UI, no KDP formatting, poor customization, no uniqueness
Opportunity	Professional KDP output alone makes you worth paying for

5.6 Competitive Positioning Summary

Competitor	Their Edge	Your Counter
KDPForge	Volume (410+ types)	Quality + real uniqueness + clean UX
BooksGenie.ai	AI word lists	Verified puzzles, no daily limits
KDP Tools Pro	Cheap lifetime	Better product, comparable pricing
Instant Puzzle Gen	Preview feature	Match preview + add Sudoku + uniqueness
Free generators	Free	Professional KDP output worth paying for

6 Marketing & User Acquisition

6.1 Early Acquisition Strategy

1. Offer free beta access to first 10–20 users — collect real feedback
2. Give beta users a permanent lifetime discount as incentive
3. Publish your own puzzle books generated by the tool as proof of quality
4. Document the build publicly — attracts early adopters and SEO

6.2 Marketing Channels

Channel	Tactic
Reddit	r/kdp, r/selfpublishing, r/passive_income — helpful posts, not ads
YouTube	Comment on KDP tutorials, create your own tutorials
Facebook	KDP publisher groups — answer questions, share the tool
SEO Blog	Target “sudoku book generator”, “kdp puzzle book” keywords
Free tool pages	Free single-puzzle generator for organic traffic and signups
Indie communities	Forums, Discord servers, self-publishing communities
Affiliate (V3)	KDP publishers have large audiences — incentivize sharing

7 Pricing Strategy

7.1 V1 — One-Time Purchase

Keep it simple for launch. No subscription friction. Let users pay once and generate.

Tier	Price	Includes
Starter	\$9 one-time	1 book, up to 100 puzzles, book-level uniqueness
Publisher	\$19 one-time	5 books, up to 500 puzzles, book-level uniqueness
Pro	\$49 one-time	Unlimited books, user-level uniqueness, all formats

7.2 V2 — Subscription (Per Puzzle Type)

Users subscribe only to the puzzle types they use. Keeps pricing fair and competitive.

Plan	Price	Includes
Sudoku Only	\$9.99/mo	Unlimited Sudoku books, user-level uniqueness
Word Search	\$9.99/mo	Unlimited Word Search books
All Puzzles	\$19.99/mo	All puzzle types, user-level uniqueness
Pro	\$39.99/mo	All types, global-level uniqueness, bulk generation

8 Quick Reference

8.1 Uniqueness Level Decision Guide

Situation	Recommended Level
User is making one book	Book-level (default, free)
User is making a series (Vol. 1–N)	User-level (paid)
User wants “guaranteed unique” marketing claim	Global-level (premium)
Two users might sell the same book on Amazon	Global-level prevents this

8.2 V1 Build Checklist

- ☐ Seed counter table with atomic increment
- ☐ Deterministic Sudoku generator (backtracking, seeded RNG)
- ☐ SHA-256 hash normalization (81-char string)
- ☐ puzzle_registry table with SKIP LOCKED reservation
- ☐ BullMQ workers for pre-generation and PDF rendering
- ☐ Redis for hash cache + reservation TTL
- ☐ KDP PDF output: 6×9, 8×10, 8.5×11 trim sizes
- ☐ Answer key pages
- ☐ Low-res browser preview before purchase
- ☐ PDF 7-day expiry + cleanup job
- ☐ LemonSqueezy payment integration
- ☐ Google + Email auth
- ☐ SEO landing page

End of Document